

타원곡선 암호에 대한 양자회로 구현동향

https://youtu.be/_qi8p9-zLOg

정보컴퓨터공학과
송경주

서론

- 타원곡선 암호 공격 시기를 평가하기 위해 쇼어 알고리즘에 동작할 ECDLP 양자회로 구현이 필요하며 이에 대한 최적화 연구들이 진행되고 있음
- 타원곡선 암호에 대한 양자회로 구현 동향 파악을 통해 양자보안 기간 산출가능
 - 즉, 기존 ECC 암호 시스템이 양자컴퓨터에 의해 언제 해독될지 예측가능
- 구현 최적화 동향 및 양자컴퓨터 개발 속도에 따라 PQC 암호로의 전환 우선순위 및 시기를 결정할 수 있음

ECDLP 양자회로 구현동향 – Prime ECC

Roetteler et al. (2017)

- 몽고메리 모듈러 곱셈 및 윈도우 스칼라 곱셈 기법 사용
- Kaliski's algorithm을 사용한 역원 양자회로 설계

(큐비트 최적화)

Haner et al. (2020)

- 윈도우 기반 곱셈과 out-of-place 연산을 결합하여 depth 개선
- ancilla 큐비트를 할당/역연산 지연(Pebbling game)기법 적용

(Toffoli/T 게이트 및
Depth 최적화)

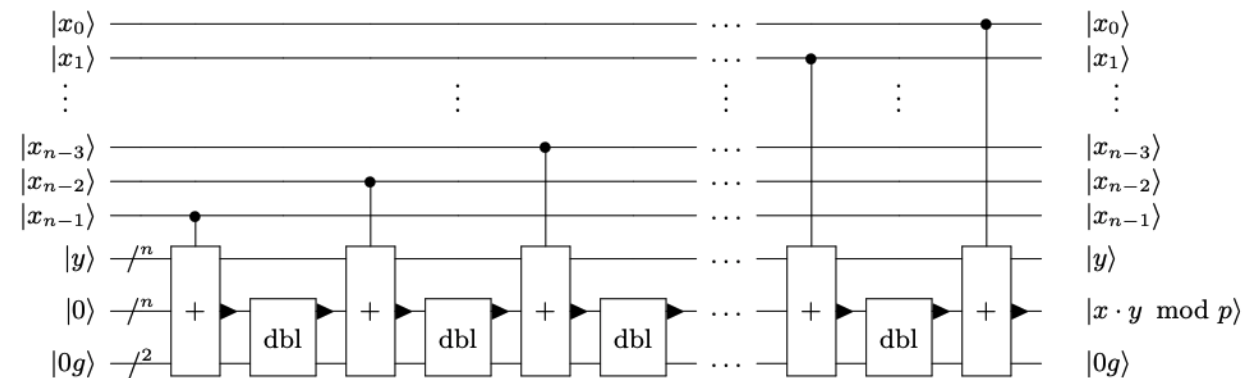
Kim et al. (2026)

- Quantum Carry Save Adder (QCSA) 을 통해 곱셈 병렬화
- 역원 연산에서 조건(control) 체크를 생략하여 회로 깊이 최소화
- 오류정정 기법을 적용했을 때의 실제 공격비용 추정

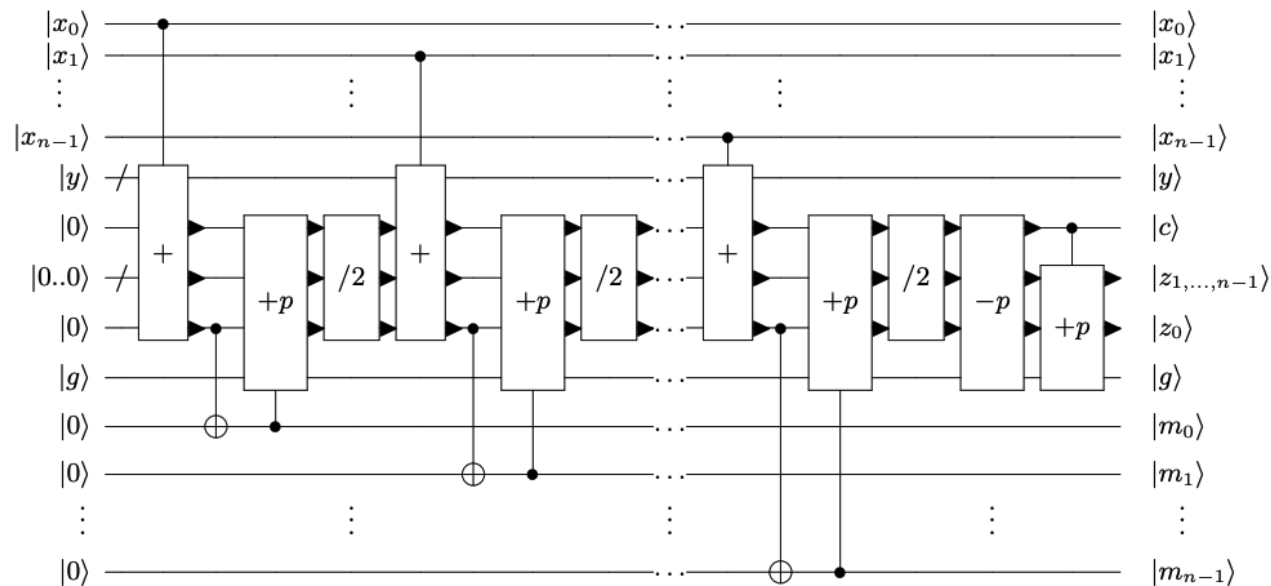
(Depth 최적화)

ECDLP 양자회로 구현동향 – Prime ECC

- Roetteler et al. (2017)
- 덧셈과 더블링의 반복으로 진행되는 모듈러 곱셈과 몽고메리 곱셈기를 각각 구현하였음
 - 일반 모듈러 곱셈: 덧셈과 더블링(덧셈 2개)으로 구성
- 몽고메리 곱셈기가 양자자원 측면에서 더욱 효율적임 → 몽고메리 곱셈기 적용



<모듈러 곱셈>

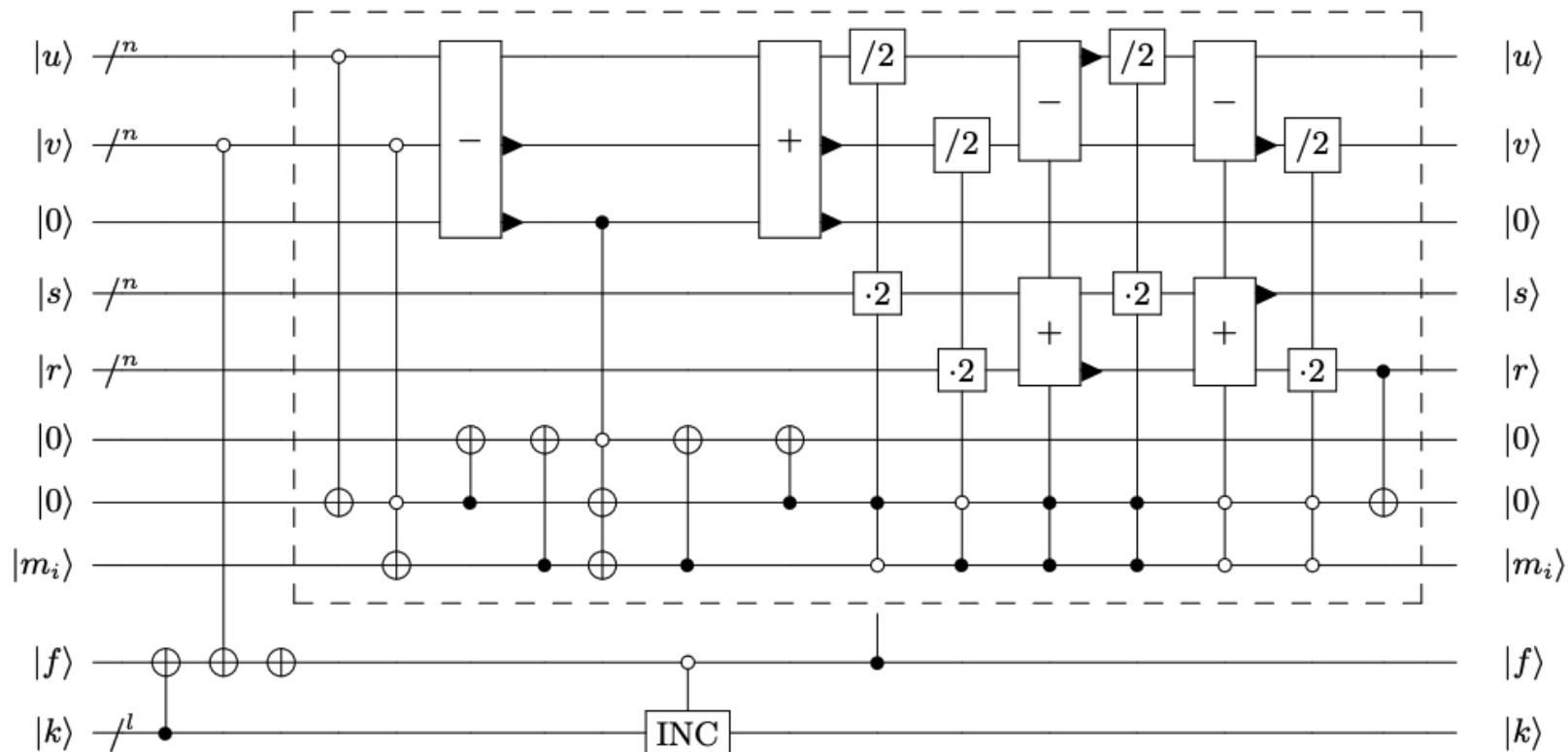


<몽고메리 모듈러 곱셈>

ECDLP 양자회로 구현동향 – Prime ECC

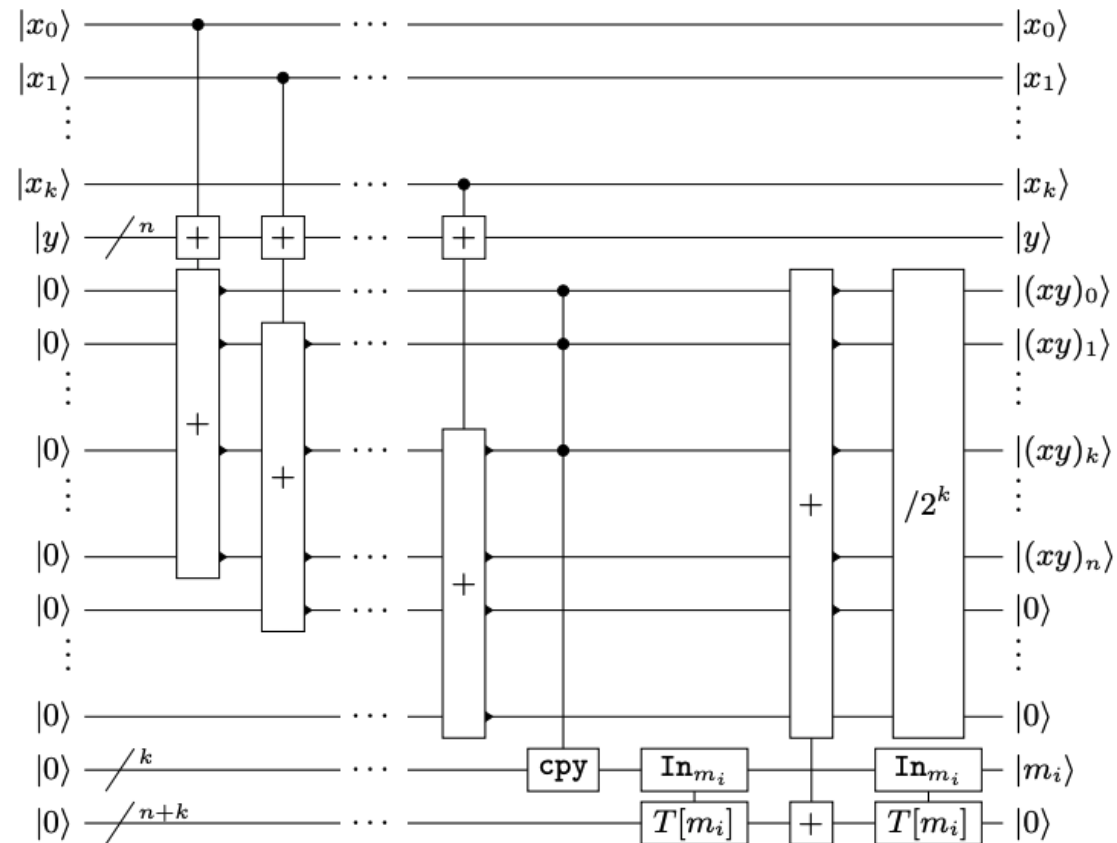
- Roetteler et al. (2017)

- Kaliski algorithm을 기반으로 역산 양자회로 설계
- 특정 큐비트 조건에 따라 회로를 멈출 수 없으므로 (가역성), worst case 만큼 $2n$ 번 반복



ECDLP 양자회로 구현동향 – Prime ECC

- Haner et al. (2020)
- 몽고메리 곱셈기에 window 기법을 적용함
- 몽고메리 모듈러 연산에 필요한 값을 table look-up 으로 미리 만들어 두고, 큐비트 상태에 따라 값을 가져와서 덧셈함
- window 사이즈에 따라 반복이 달라짐: $\lceil n/k \rceil$



ECDLP 양자회로 구현동향 – Prime ECC

• Haner et al. (2020)

- 기존 Kaliski's algorithm을 SWAP게이트를 사용한 양자회로의 조건식으로 수정하여 이전 연구 (Roetteler et. al) 대비 연산 수를 줄임

(a) Kaliski's algorithm

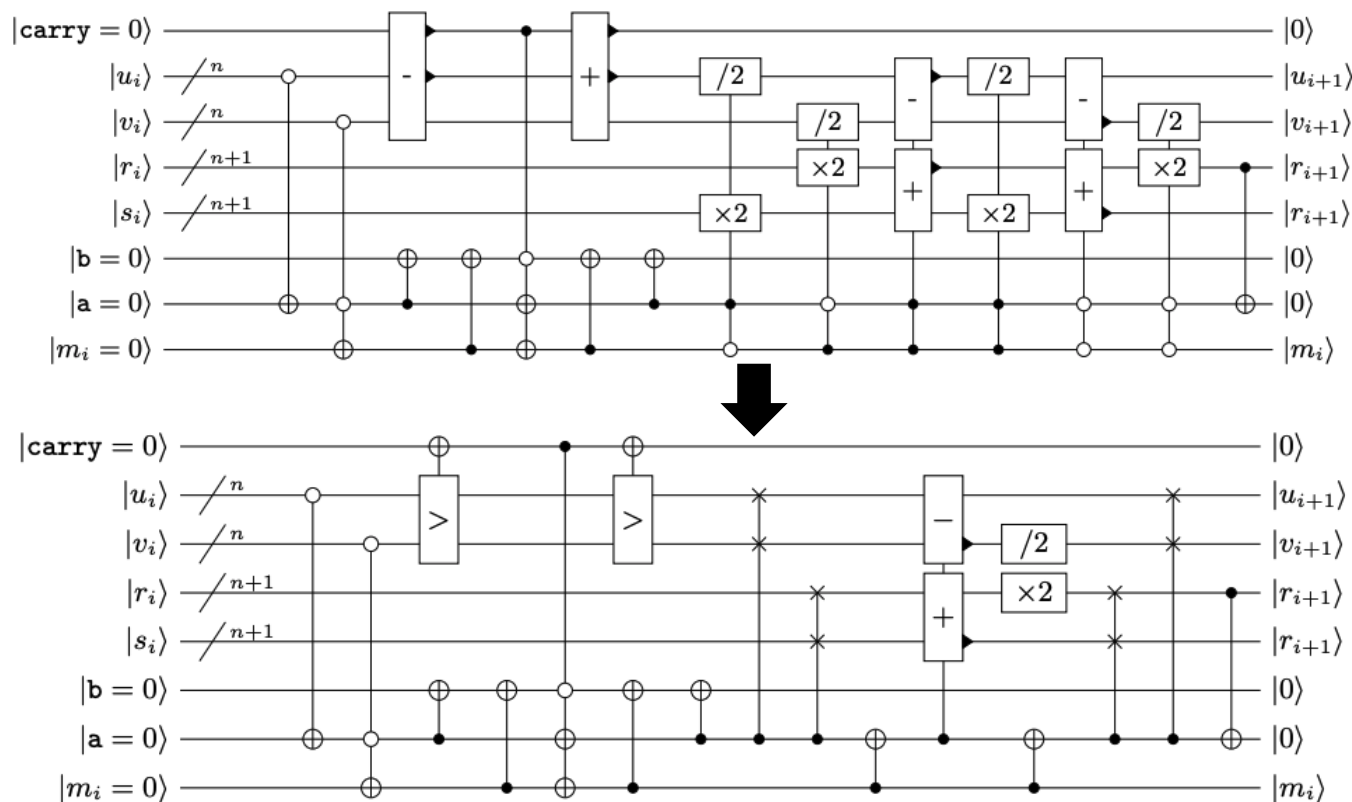
```

1: if  $u$  odd and  $v$  even then
2:    $v \leftarrow v/2$ 
3:    $r \leftarrow 2r$ 
4: else if  $u$  even and  $v$  odd then
5:    $u \leftarrow u/2$ 
6:    $s \leftarrow 2s$ 
7: else if  $u$  odd and  $v$  odd and  $u > v$ 
   then
8:    $u \leftarrow (u - v)/2$ 
9:    $r \leftarrow r + s$ 
10:   $s \leftarrow 2s$ 
11: else if  $u$  odd and  $v$  odd and  $v \geq u$ 
   then
12:   $v \leftarrow (v - u)/2$ 
13:   $s \leftarrow r + s$ 
14:   $r \leftarrow 2r$ 
15: end if
  
```

(b) Equivalent formulation

```

1:  $b_{\text{swap}} \leftarrow \text{false}$ 
2: if  $u$  even and  $v$  odd, or  $u$  and  $v$  both
   odd and  $u > v$  then
3:   swap  $u$  and  $v$ 
4:   swap  $r$  and  $s$ 
5:    $b_{\text{swap}} \leftarrow \text{true}$ 
6: end if
7: if  $u$  odd and  $v$  odd then
8:    $v \leftarrow v - u$ 
9:    $s \leftarrow r + s$ 
10: end if
11:  $v \leftarrow v/2$ 
12:  $r \leftarrow 2r$ 
13: if  $b_{\text{swap}}$  then
14:   swap  $u$  and  $v$ 
15:   swap  $r$  and  $s$ 
16: end if
  
```



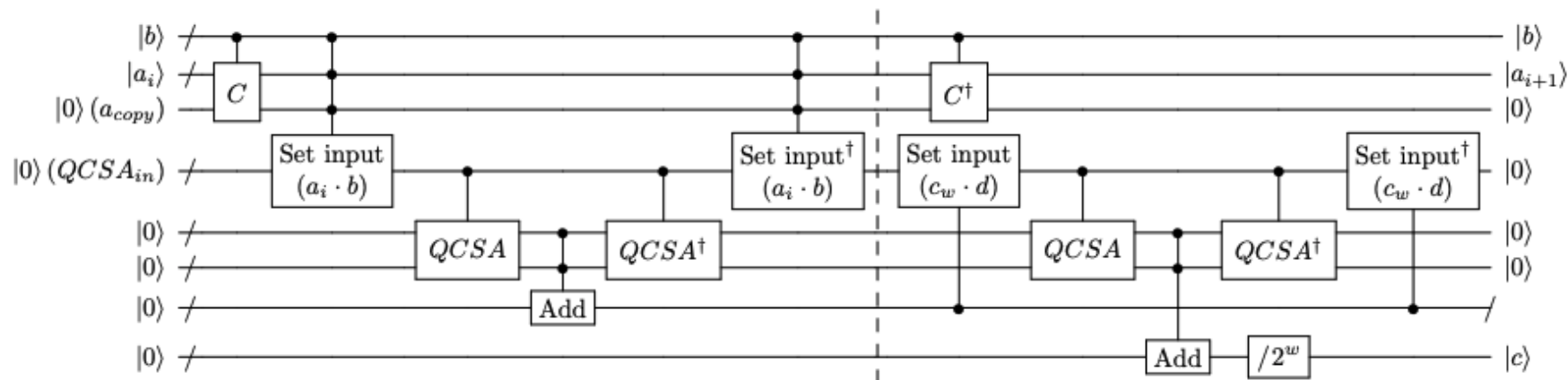
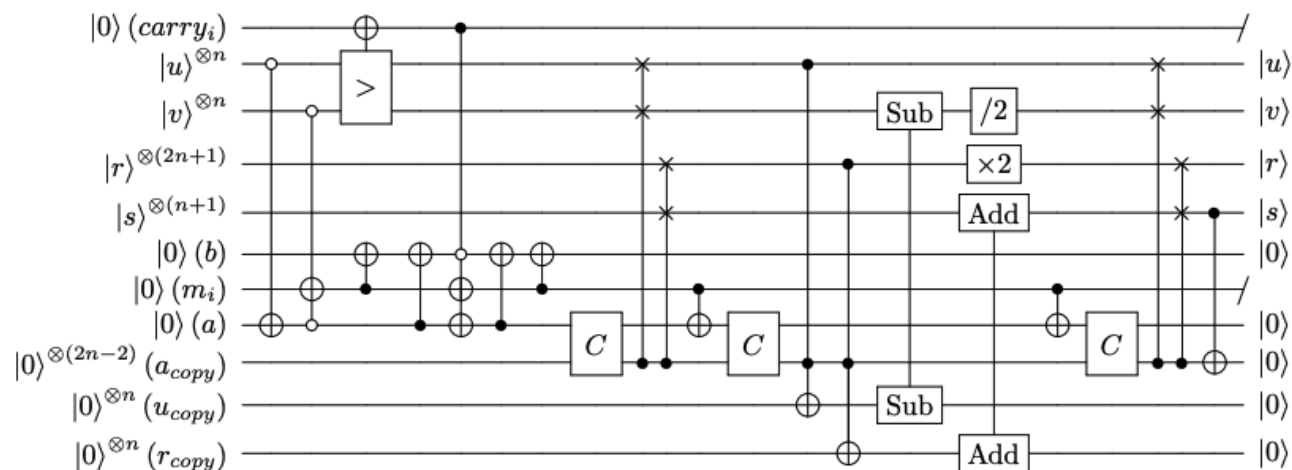
ECDLP 양자회로 구현동향 – Prime ECC

- Kim et al. (2026)

- 모듈러 곱셈 및 역산에 Quantum Carry Save Adder (QCSA) 적용

- QCSA는 순차적으로 carry 값을 적용하지 않고 다중 피연산자에 대해 병렬로 덧셈 가능

- QCSA를 적용하기 위해 input을 셋팅 하는 단계가 추가로 설계되었음



ECDLP 양자회로 구현동향 – Prime ECC

- 자원 추정 결과

Ref	큐비트	CNOT	T	T-depth	Full Depth
Roetteler et al.	1754	N/A		48600000000	N/A
Haner et al. (Qubit opt)	2080	108,905,241	82,304,057	36,606,886	85,616,254
Haner et al. (Depth opt)	3341	581,960,676	208,167,629	1,903,404	7,986,843
Kim et al. (Qubit opt)	6476	230,834,330	122,440,398	943,529	2,438,870
Kim et al. (Depth opt)	23729	173,988,372	69,652,392	320,949	1,076,644

ECDLP 양자회로 구현동향 – Binary ECC

Putranto et al. (2022)

- 개선된 Karatsuba 곱셈기 사용
- FLT 기반 역원 회로를 설계

(큐비트 최적화)

Taguchi et al. (2023)

- FLT 기반 역원 연산 + 덧셈사슬 구조 접목
- 덧셈 사슬 특성이 양자 자원에 어떤 영향을 미치는지 확인

(큐비트 및 Depth 최적화)

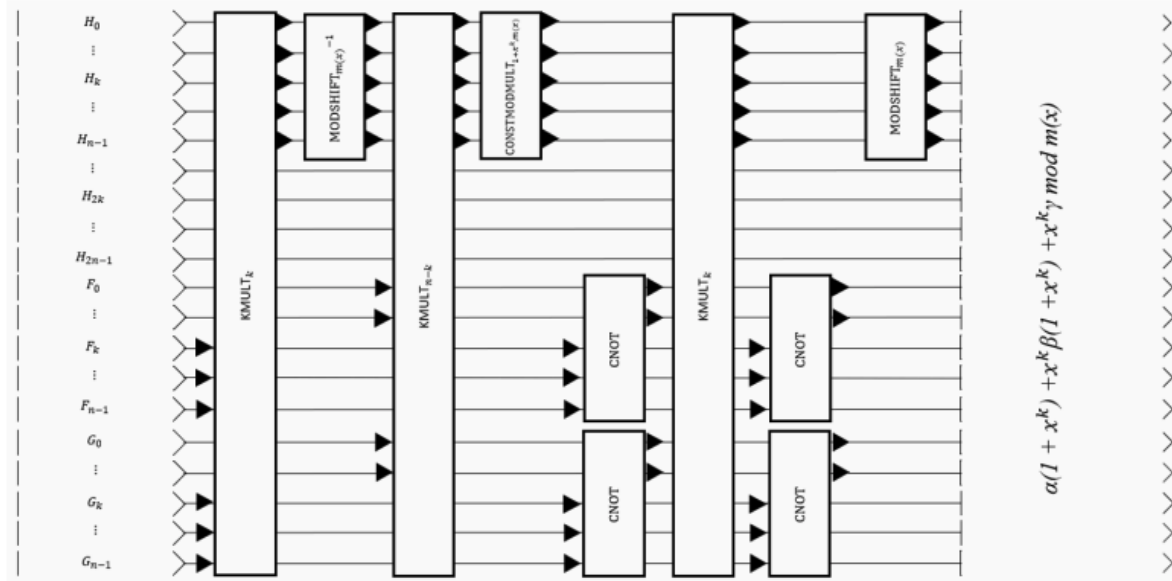
Jang et al. (2025)

- 낮은 depth의 Karatsuba 곱셈기 적용
- Out-of-place 구조의 점덧셈 구조 설계
- NIST 양자보안 기준에 따른 분석

(Depth 최적화)

ECDLP 양자회로 구현동향 – Binary ECC

- Putranto et al. (2022)
- 개선된 Karatsuba 곱셈기 사용-기존 방식의 모듈러 곱셈 단계를 수정 (역연산을 제거)
 - 연산 단계를 재배치하여 특정 상수항($1 + x^k$)을 지우기 위한 역연산 제거
- FLT 기반 역산 알고리즘에 개선된 Karatsuba 곱셈기 적용



Algorithm 2 INV_Imp. Our proposed algorithm for FLT-based inversion, which modifies [BBvHL21, LPWK22]. For simplicity, we follow the notation style of [BBvHL21]

Fixed input : A constant field polynomial $m(x)$ of degree $n > 0$. $k_1 > k_2 > \dots k_t \geq 0$ such that $\sum_{s=1}^3 2^{k_s} = n - 1$. $k_{max} = 2 * k_1 + t$.

Quantum input :

- A non-zero binary polynomials of degree up to $n - 1$ stored in array (register)

f_0 of size n to invert.

- k zero arrays of size n initialized to an all- $|0\rangle$ state: $f_1, \dots, f_k$.

Result : inverse of the input, stored in f_k

```

1: for  $i = 1, \dots, k_1$  do //stage 1
2:   CNOT( $f_{2*(i-1)+1}, f_{2*(i-1)}$ )
3:   for  $i = 1, \dots, t-1$  do
4:     SQUARE( $f_{2*(i-1)+1}$ )
5:     MODMULT_Imp( $f_{2*(i-1)+2}, f_{2*(i-1)+1}, f_{2*(i-1)}$ )
6: for  $s = 1, \dots, t-1$  do //stage2
7:   for  $k = 1, \dots, 2^{k_s+1}$  do
8:     SQUARE( $f_{2*(i-1)+1}$ )
9:     MODMULT_Imp( $f_{2*(i-1)+2}, f_{2*(i-1)+1}, f_{2*(i-1)}$ )
10:  if  $t = 1$  then
11:    swap ( $f_{k_1}, f_k$ )
12:  SQUARE( $f_k$ ) //stage 3

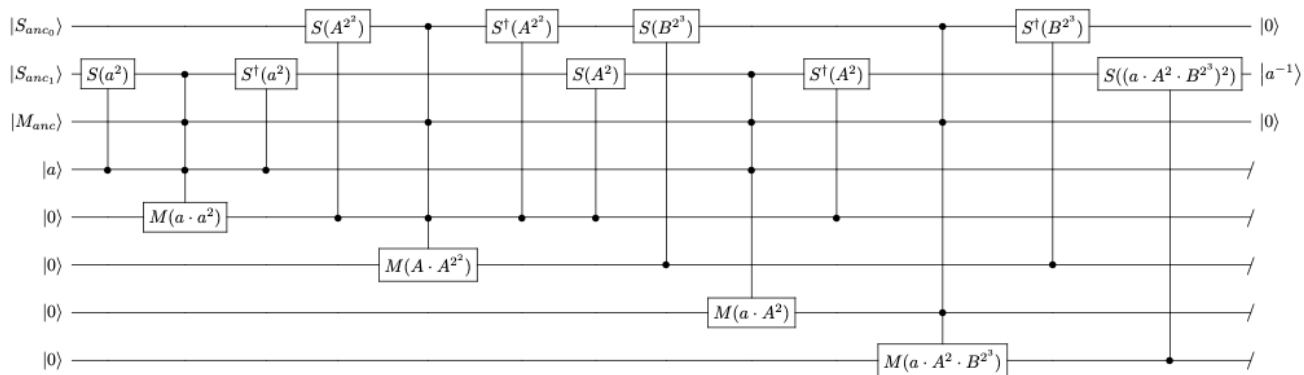
```

ECDLP 양자회로 구현동향 – Binary ECC

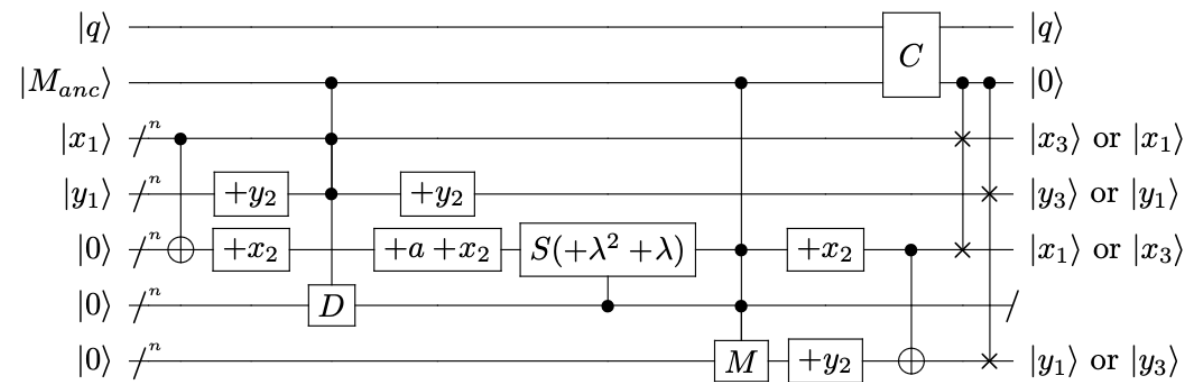
- Taguchi et al. (2023)
- 기존 FLT 기반 알고리즘은 고전적인 Itoh-Tsujii(IT) 방식의 고정된 지수 연산 순서를 그대로 양자 회로로 구현
- FLT 기반의 양자 역원 연산에 덧셈 사슬(Addition chain) 구조 접목
 - FLT 역원 연산(f^{2^n-2})의 핵심인 $f^{2^{n-1}-1}$ 계산을 $n-1$ 에 대한 임의의 덧셈 사슬로 수행 가능함을 수학적으로 보임
 - 덧셈 사슬의 구성 요소인 Doubled term(d)과 Added term(m)의 비율에 따라 양자 자원 소비가 달라짐을 분석

ECDLP 양자회로 구현동향 – Binary ECC

- Jang et al. (2025)
- 가장 낮은 depth의 Karasuba 곱셈기를 사용함
- 역산 연산에 out-of-place 의 제곱 및 곱셈을 단계적으로 설계하여 depth를 최소화 하였음
- out-of-place 구조의 점덧셈 구성 (연산 수를 줄임)



Low-depth 역산



Out-of-place 점덧셈

ECDLP 양자회로 구현동향 – Binary ECC

- 자원 추정 결과

Ref	큐비트	CNOT	T	T-depth	Full Depth
Putranto et.al. [9]	6226	11434619	605,367	1720152	N/A
Taguchi et.al. [11] (Basic)	4812	11428959	605,367	1720132	N/A
Taguchi et.al. [11] (Extended)	3963	11454547	605,367	1727164	N/A
Jang et.al. [12]	144672	7856986	534762*7	10957	11556

Q & A